

# 이미지로 확장하는 멀티모달 RAG

# 학습 체크리스트 (1/2)

---

☐ 멀티모달·Base64·OCR의 개념과 Image RAG에서 맡는 역할 이해

☐ `Media`를 첨부한 `ChatClient` 호출로 캡션·태그·OCR을 추출하는 방법 습득

☐ 이미지 검증·리사이즈와 타일 기반 토큰 비용 주의점 파악

☐ 해석 텍스트와 이미지 벡터를 함께 저장하는 이중 구조 이해

# 학습 체크리스트 (2/2)

---

☐ `gemini-embedding-2`를 OpenAI 호환 엔드포인트로 호출하는 설정 습득

☐ 이미지 전용 `PgVectorStore`와 메타데이터 스키마 설계 방법 파악

☐ Data URI로 임베딩 입력을 바꾸는 어댑터의 역할 이해

☐ 크로스모달 검색과 소유자 필터·임계값 튜닝 방법 습득

# 텍스트만 아는 모델

---

- 입력 형태 한계: 405-1까지 쓴 모델은 토큰화된 문자열만 입력으로 받음
- 이미지 정보 전달 불가: 이미지 속 글자·형태·수치는 텍스트 프롬프트로 옮길 수 없음
- 검색·답변의 전제: 이미지를 근거로 쓰려면 모델이 이미지를 직접 볼 수 있어야 함

# 멀티모달이란

---

## 멀티모달 (MultiModal)

이미지·오디오·비디오 같은 다양한 형태의 데이터를 텍스트와 함께 하나의 프롬프트에 담아 처리하는 방식

- **입력 확장:** 텍스트 전용 프롬프트에 다른 데이터 형태가 추가됨
- **처리 주체:** 모델이 각 모달리티를 함께 해석해 하나의 응답을 생성

# Spring AI가 미디어 데이터를 다루는 방법

---

- 추상화 단위: 미디어 데이터를 `Media` 객체로 감싸 표현
- 구성 요소: `MimeType` (모달리티 종류)과 실제 콘텐츠(`Resource` 또는 URI)로 구성
- 전달 경로: `UserMessage` 에 붙여 멀티모달 모델로 전달

# Base64 인코딩

---

## Base64 인코딩 (Base64 Encoding)

바이너리 데이터를 출력 가능한 64개 ASCII 문자로 바꾸는 이진-텍스트 인코딩

- 문자 구성: A-Z, a-z, 0-9, +, / 와 패딩 문자 `=`
- 필요한 이유: HTTP JSON이나 HTML은 바이너리 바이트를 직접 다루기 어려움

# Data URI 한 줄 뜯어보기

---

1 `data:image/png;base64,iVBORw0KGgoAAAANSUhEUg ...`

2

3 구조: `data:{mimeType};base64,{payload}`



# Base64는 공짜가 아니다

---

- 크기 증가: 3바이트 바이너리가 4바이트 ASCII로 바뀌어 원본 대비 약 33% 증가
- 페이로드 부담: 요청 페이로드·메모리 사용량이 함께 급증
- 토큰 비용: 인코딩된 텍스트만큼 모델 토큰 사용량도 늘어남
- 대응 원칙: 인코딩 전 이미지 리사이즈가 필수

# OCR이란

---

광학 문자 인식 (OCR, Optical Character Recognition)

이미지 속 글자를 디지털 텍스트로 추출하는 기술

- **적용 대상:** 문서·표지판·영수증·차트 수치 등 이미지 속 문자 정보

# OCR이 검색 품질을 좌우한다

---

- 캡션의 한계: 시각적 특징이나 요약 캡션만으로는 세부 문자가 검색에서 누락됨
- 누락 예시: 제품 모델명·도로명·그래프 수치 같은 텍스트가 대표적
- 저장 전략: OCR 결과를 `ocrText`로 함께 저장하면 근거 정확도가 올라감
- 도구 선택: 별도 OCR 라이브러리 없이 Gemini 모델이 캡션·태그와 함께 추출

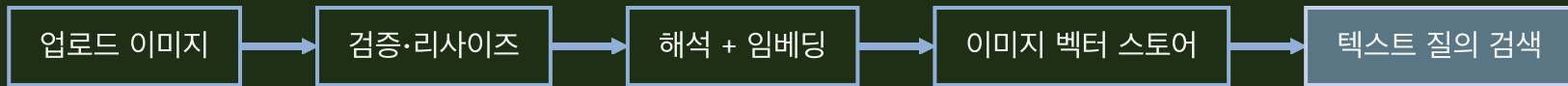
# 이번 차시에서 만들 것

---

- 흐름 계승: 404의 이미지 업로드 폼과 405-1의 RAG 파이프라인을 이어받음
- 두 가지 산출물: 이미지 한 장에서 해석 텍스트와 이미지 벡터를 함께 저장
- 해석 텍스트 용도: 캡션·태그·OCR을 묶어 근거 제시와 답변 생성에 사용
- 이미지 벡터 용도: 멀티모달 임베딩으로 텍스트↔이미지 크로스모달 검색에 사용

# 업로드에서 답변까지

---



# 이미지를 모델에 전달하는 두 가지 길

---

|       |                   |                 |
|-------|-------------------|-----------------|
| 항목    | 인라인(Base64·바이트)   | URL·File API 참조 |
| 난이도   | 낮음, 바로 전송         | 별도 업로드 필요       |
| 크기 제한 | 텍스트+인라인 합계 약 20MB | 더 큰 파일 가능       |
| 재사용성  | 매 요청 재전송          | 여러 요청에서 참조      |

- 이번 차시 방향: 인라인 방식만 사용

# 해석 결과를 얻기까지

---

- 업로드 수신: `multipart/form-data`로 이미지 수신(404차시와 동일)
- 형식 검증: 확장자·Content-Type이 아니라 실제 바이트 시그니처로 검증
- Media 생성: 리사이즈·재인코딩 후 `ByteArrayResource`로 감싸 `Media` 생성
- 프롬프트 결합: `ChatClient`의 `user(...)`에서 `text(...)`와 `media(...)`를 함께 지정
- 결과 수신: 캡션·태그·OCR로 분리해 `record`로 수신

# 바이트를 Media로 감싸기

---

```
1  MimeType verifiedMimeType = MimeTypeUtils.parseMimeType(detectedMimeType);  
2  Media media = new Media(verifiedMimeType, new ByteArrayResource(resizedImageBytes));
```



# 텍스트와 이미지를 함께 프롬프트에 실기

---

```
1 ImageInterpretation result = chatClient.prompt()
2   .user(u → u.text(interpretPrompt).media(media))
3   .call()
4   .entity(ImageInterpretation.class);
```

# 해석 결과를 담은 record

---

```
1 // caption: 한 문장 설명
2 // tags: 명사 3~5개
3 // ocrText: 이미지 속 문자, 없으면 빈 문자열
4 public record ImageInterpretation(
5     String caption,
6     List<String> tags,
7     String ocrText
8 ) {}
```

# 검색에 걸리는 캡션 만들기

---

- 명사 중심 지시: 주관적 표현 대신 객체·장소·행동·색상 같은 명사를 뽑도록 지시
- OCR 규칙: `ocrText` 는 이미지 속 글자를 정확히 읽도록 규칙 명시
- 필드별 규칙: 필드별 규칙을 프롬프트에 적을수록 후속 검색 품질이 안정됨
- 기능 독립성: 멀티모달 호출과 구조화 출력(`.entity`)은 서로 독립적인 기능

# 이미지 토큰은 타일 단위로 계산된다

---

- 소형 이미지: 가로·세로 모두 384px 이하이면 258토큰 고정
- 대형 이미지: 그보다 크면 768×768 타일로 나눠 타일당 258토큰 합산
- 비용 급증: 고해상도 원본을 그대로 보내면 토큰과 인라인 Base64 비용이 함께 급증
- 대응: 리사이즈 필수(실습 기준 최대 변 1536px)

# 업로드 이미지 검증하기

---

- 매직 넘버 검사: 클라이언트가 보낸 Content-Type 헤더를 신뢰하지 않고 파일 바이트로 판별
- 디코딩 전 확인: 작은 압축 파일도 디코딩 후 메모리를 크게 차지하므로 `width × height` 상한 검사
- 손상 파일 거부: 손상·디코딩 불가 이미지는 거부
- 재인코딩 사용: 이후 단계에는 리사이즈 후 재인코딩한 바이트를 사용

# 모델 응답을 믿지 못하는 순간들

---

- 안전 필터: 응답이 거부되거나 빈 값일 수 있으며, 이때는 벡터 저장을 진행하지 않음
- 비결정성: 같은 이미지도 캡션·태그가 매번 달라짐
- 키 사용 금지: 캡션을 유일 키나 중복 판정 기준으로 쓰지 않음
- 지연·한도: 이미지 해석은 텍스트보다 지연이 길고 429 일일 한도(RPD)에 닿기 쉬움

# 해석 텍스트는 원본보다 위험하다

---

- 개인정보 노출: 얼굴·신분증이 해석 텍스트나 로그에 그대로 남지 않도록 저장 정책 수립
- 유출 용이성: 해석 텍스트는 원본 이미지보다 검색·유출이 훨씬 쉬운 형태
- 사전 기준: 저장 전에 마스킹·거부 기준을 정해둬

# 이미지를 검색 가능하게 만드는 두 갈래

---

| 방식         | 원리                           | 장점                            | 한계                           |
|------------|------------------------------|-------------------------------|------------------------------|
| 해석 텍스트 임베딩 | 캡션·OCR을 만들어 텍스트 임베딩          | 405-1 파이프라인 재사용,<br>근거 인용 쉬움  | 캡션이 놓친 색상·구도·<br>미세 객체는 못 찾음 |
| 멀티모달 임베딩   | 이미지를 그대로 임베딩<br>(텍스트도 같은 공간) | 시각 정보 손실 적고 이미지→<br>이미지 검색 가능 | 별도 모델·벡터 공간 관리<br>필요         |



# 두 산출물을 함께 저장하는 이유

---

- **해석 텍스트만 저장:** 설명은 되지만 캡션이 놓친 이미지는 찾을 수 없음
- **이미지 벡터만 저장:** 찾을 수는 있지만 근거를 문장으로 설명하지 못함
- **역할 분담:** 해석 텍스트는 답변 근거, 이미지 벡터는 검색을 담당
- **함께 저장:** 두 산출물을 한 레코드에 묶어야 검색과 설명이 모두 성립

# gemini-embedding-2

---

## 제미나이 임베딩 2 (gemini-embedding-2)

텍스트·이미지·비디오·오디오·PDF를 하나의 벡터 공간(joint embedding space)에 매핑하는 Gemini 멀티모달 임베딩 모델

- **같은 공간:** 사진과 그 사진을 설명한 문장이 가까운 벡터가 됨
- **직접 검색:** 텍스트 질의 벡터로 이미지를 바로 검색 가능
- **매핑 불필요:** 모달리티 간 별도 매핑 테이블이 필요 없음

# 출력 차원을 1536으로 맞추는 이유

---

- 선택 범위: 출력 차원은 128~3072에서 선택 가능(권장 768/1536/3072)
- 동일한 제약: 405-1의 텍스트 임베딩과 같은 이유(pgvector HNSW 2,000차원 제한)
- 실습 값: 이미지도 1536차원으로 통일해 인덱싱 가능하게 유지
- 스키마 단순화: 텍스트 벡터와 차원까지 같아져 스키마 관리가 단순해짐

# 모델 사양과 한도

---

| 항목        | 값                  |
|-----------|--------------------|
| 출력 차원     | 128~3072 (실습 1536) |
| 요청당 이미지 수 | 최대 6장              |
| 지원 형식     | PNG, JPEG          |
| 입력 토큰 한도  | 전체 모달리티 합산 8,192   |

# 어떻게 호출할 것인가

---

| 선택지                   | 특징                                                                                                      |
|-----------------------|---------------------------------------------------------------------------------------------------------|
| OpenAI 호환<br>엔드포인트    | Gemini <code>/embeddings</code> 를 Spring AI OpenAI 스타터로 호출, 표준 <code>EmbeddingModel</code> 추상화·차원 선택 가능 |
| Vertex AI 멀티모달<br>임베딩 | 프레임워크 통합되나 1408차원 고정·GCP 자격 증명 필요                                                                       |
| 직접 REST 호출            | 최신 파라미터 가능하나 통신·오류 처리를 직접 구현                                                                            |

- 405-1에서 쓴 Google GenAI 임베딩 모듈은 아직 텍스트 전용이라 OpenAI 호환 엔드포인트를 택함

# Data URI가 곧 이미지 입력이다

---

- **입력 형태:** OpenAI 임베딩 API의 `input` 은 문자열 배열( `List<String>` )
- **이미지 인식:** 입력 문자열이 `data:{mimeType};base64,{payload}` 포맷이면 이미지로 인식
- **동일한 반환 형태:** 반환 벡터는 텍스트 임베딩과 완전히 동일한 float 배열
- **혼합 요청:** 텍스트와 이미지를 같은 배열에 섞어 한 번에 요청 가능

# 임베딩 요청 본문

---

```
1  POST /v1beta/openai/embeddings
2
3  {
4    "model": "gemini-embedding-2",
5    "input": [
6      "노을 지는 해변 사진",
7      "data:image/png;base64,iVBOR ..."
8    ],
9    "dimensions": 1536
10 }
```

# 이미지 벡터 한 줄로 얻기

---

```
1 String dataUri = "data:%s;base64,%s"  
2     .formatted(mimeType, Base64.getEncoder().encodeToString(imageBytes));  
3  
4 float[] imageVector = imageEmbeddingModel.embed(dataUri);
```



# 의존성 추가

---

```
1 // 멀티모달 채팅은 405-1의 google-genai 스타터로 충분, 임베딩용으로 OpenAI 스타터 추가
2 implementation("org.springframework.ai:spring-ai-starter-model-openai")
3 implementation("org.springframework.ai:spring-ai-starter-vector-store-pgvector")
```

# Gemini 임베딩 엔드포인트 설정

---

```
1  spring:
2    ai:
3      model:
4        embedding: google-genai
5      openai:
6        api-key: ${GOOGLE_AI_API_KEY}
7        embedding:
8          base-url: https://generativelanguage.googleapis.com/v1beta/openai
```

# 설정에서 걸려 넘어지는 지점

---

- `base-url` 경로: 버전 경로(`v1beta/openai`)까지 직접 포함해야 함(스타터가 뒤에 `/embeddings`를 붙임)
- 기본 모델 유지: 기본 `EmbeddingModel`은 405-1의 텍스트용을 유지하고 이미지용은 별도 빈으로 명시 생성
- 업로드 크기 상한: multipart 업로드 크기 상한(5MB/10MB)을 설정
- RAG 값 분리: `top-k` · `similarity-threshold`는 `app.ai.image-rag`로 따로 둬م(임곗값 0.5 전후)

# 이미지 전용 벡터 테이블

---

```
1 image_vector_store
2   ├── id          uuid          PK
3   ├── content     text          미디어 Document이므로 비어 있을 수 있음
4   ├── metadata    json          해석 텍스트·객체 키·소유자
5   ├── embedding   vector(1536)  이미지 멀티모달 임베딩
6   + HNSW 인덱스 (코사인 거리)
```

# 테이블을 나누는 이유

---

- 모델 불일치: 서로 다른 모델의 벡터가 섞이면 유사도 계산이 무의미해짐
- 405-1 재사용 불가: 텍스트용 `vector_store` 를 그대로 쓰지 않음
- 인덱스 튜닝 차이: 모달리티별로 HNSW 파라미터 최적점이 다름
- 필터링 전략 차이: 이미지는 객체 키·태그 중심 필터가 별도로 필요

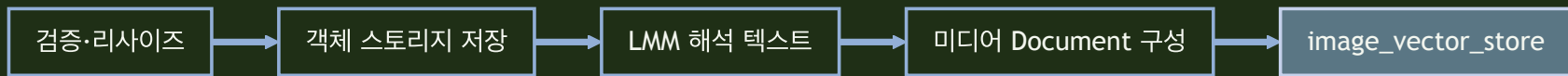
# 메타데이터에 담을 것

---

| 항목                          | 설명               |
|-----------------------------|------------------|
| <code>objectKey</code>      | 스토리지 원본 키        |
| <code>interpretation</code> | 캡션·태그·OCR 근거 텍스트 |
| <code>tags</code>           | 필터·재정렬용          |
| <code>ownerId</code>        | 멀티테넌트 보안 필터, 필수  |
| <code>contentHash</code>    | 중복 판정            |

# 업로드 한 장이 저장되기까지

---



# 임베딩 입력을 이미지로 바꾸는 어댑터

---

```
1  @Override
2  public String getEmbeddingContent(Document document) {
3      Media media = document.getMedia();
4      if (media == null) {
5          return document.getText();
6      }
7      String base64 = Base64.getEncoder()
8          .encodeToString(media.getDataAsByteArray());
9      return "data:%s;base64,%s".formatted(media.getMimeType(), base64);
10 }
```



# 이미지 전용 벡터 스토어 등록

---

```
1  @Bean("imageVectorStore")
2  public VectorStore imageVectorStore(
3      JdbcTemplate jdbcTemplate,
4      @Qualifier("imageApiEmbeddingModel") EmbeddingModel imageEmbeddingModel) {
5      return PgVectorStore.builder(jdbcTemplate, imageEmbeddingModel)
6          .vectorTableName("image_vector_store")
7          .dimensions(1536)
8          .initializeSchema(true)
9          .build();
10 }
```

# 해석 텍스트와 이미지를 한 Document로

---

```
1 Document imageDocument = Document.builder()
2     .media(media)
3     .metadata("interpretation", interpretationText)
4     .metadata("objectKey", objectKey)
5     .metadata("ownerId", currentUserId.toString())
6     .build();
7
8 imageVectorStore.add(List.of(imageDocument));
```

# 저장 단계에서 조심할 것

---

- **체크 분할 금지:** 이미지 한 장은 그 자체로 검색 단위
- **원본 저장 위치:** DB에 `bytea` 로 넣지 않고 스토리지 키만 보관
- **Data URI 범위:** 임베딩 요청에만 쓰고 메타데이터에 남기지 않음
- **빈 구분:** 텍스트·이미지 `EmbeddingModel` 빈이 둘이므로 `@Qualifier` 명시
- **실패 시 롤백:** 해석·임베딩·DB 저장 실패하면 업로드한 객체를 삭제

# 텍스트 질의로 이미지 찾기

---

```
1 SearchRequest request = SearchRequest.builder()
2     .query(question)
3     .topK(aiProperties.imageRag().topK())
4     .similarityThreshold(aiProperties.imageRag().similarityThreshold())
5     .filterExpression(ownerFilter)
6     .build();
7
8 List<Document> results = imageVectorStore.similaritySearch(request);
```

# 검색 결과로 답변을 만드는 두 방식

---

| 방식            | 비용·속도 | 특징                                                    |
|---------------|-------|-------------------------------------------------------|
| 해석 텍스트<br>재사용 | 저렴·빠름 | 저장해 둔 캡션·태그·OCR을 텍스트 Document로 바꿔 프롬프트에 주입, 캡션 품질에 종속 |
| 이미지 원본<br>재첨부 | 비쌈·느림 | 스토리지 원본을 <code>Media</code> 로 붙여 직접 해석하므로 정확          |

- 1차는 해석 텍스트로 답하고 필요할 때만 원본을 첨부

# 검색 품질 튜닝

---

- 필터 구성: 소유자 필터는 문자열 보간 대신 `FilterExpressionBuilder`로 구성
- 임곗값 재측정: 점수 분포가 텍스트 RAG와 달라 0.7을 쓰면 검출률 저하, 0.5 전후로 재측정
- 2단계 필터링: `topK`를 넉넉히 뽑은 뒤 임곗값·메타데이터로 걸러냄
- 하이브리드 보완: 이미지 벡터 검색과 해석 텍스트(OCR) 검색 결과를 합쳐 재정렬

# 학습 요약 (1/2)

---

- 멀티모달: 이미지를 `Media`로 감싸 텍스트와 함께 하나의 프롬프트에 실어 보냄
- Base64: 바이너리를 Data URI 텍스트로 바꾸는 인코딩, 약 33% 커지므로 리사이즈 선행
- OCR: 이미지 속 문자를 `ocrText`로 남겨 검색과 근거의 정확도를 끌어올림
- 이미지 해석: `.entity(...)`로 캡션·태그·OCR을 `ImageInterpretation` record에 담음

# 학습 요약 (2/2)

---

- `gemini-embedding-2`: 텍스트와 이미지를 같은 1536차원 공간에 투영하는 멀티모달 임베딩
- 호출 경로: Google GenAI 모듈이 이미지 임베딩을 아직 지원하지 않아 OpenAI 호환 `/embeddings` 사용
- 이중 저장: 해석 텍스트는 메타데이터에, 이미지 벡터는 이미지 전용 `PgVectorStore`에 저장
- 크로스모달 검색: 텍스트 질문으로 이미지를 찾고, `ownerId` 필터와 임계값 재측정으로 품질을 잡음